

Data Analysis using Pandas(1).ipynb

Cell 1 (markdown)

Introduction to Data Analysis

Cell 2 (code)

```
#Data Analysis:
"""Data Analysis is the process of collecting, cleaning, transforming,
and interpreting data to extract meaningful insights and support decision-making."""
```

Cell 3 (code)

```
#In simple terms:
#Raw Data → Process → Insights → Decisions
```

Cell 4 (code)

```
#Key Steps in Data Analysis:
#Data Collection
#Data Cleaning (handling missing/incorrect data)
#Data Transformation
#Data Analysis & Visualization
#Interpretation & Decision Making
```

Cell 5 (code)

```
#Types of Data
#Understanding data types is crucial before working with tools like Pandas.
```

Cell 6 (code)

```
#1.Structured Data:
"""Organized in rows and columns
Easily stored in databases or spreadsheets
Examples:
Excel sheets
SQL databases
Tools used:
Microsoft Excel
MySQL
"""
```

Cell 7 (markdown)

Cell 8 (code)

```
#2.Unstructured Data:
"""No fixed format or structure
Difficult to analyze directly
```

Examples:
Images, videos
Text documents, social media posts
Requires preprocessing before analysis"

Cell 9 (markdown)

Cell 10 (markdown)

■ Introduction to Pandas

Cell 11 (code)

```
# Pandas is a powerful Python library used for data manipulation and analysis.
```

Cell 12 (code)

```
"""Key Features:  
Handles structured data efficiently  
Provides DataFrame (table-like structure)  
Supports data cleaning, filtering, grouping  
Integrates weall with visualization libraries
```

```
Core Data Structures:  
Series → 1D array (like a column)  
DataFrame → 2D table (like Excel sheet)
```

Cell 13 (code)

```
import pandas as pd  
print(pd.__version__)
```

Cell 14 (code)

```
#Microsoft Excel → Best for beginners  
#MySQL → Best for databases  
#Pandas → Best for programming-based analysis
```

Cell 15 (code)

```
#Real World Use Cases of Pandas:  
"""
```

1. Business Analytics
Sales analysis
Customer behavior tracking
Revenue forecasting
2. Healthcare
Patient data analysis
Disease prediction
Medical research
3. Finance
Stock market analysis

Risk assessment
Fraud detection
4. E-commerce
Product recommendation
Sales trends
Inventory management
""

Cell 16 (code)

#Series vs DataFrame

Cell 17 (markdown)

![image.png](attachment:b9ee2541-9929-42fa-82de-3f5e735db85f.png)

Cell 18 (code)

import pandas as pd

```
# Creating a Series
data = [100, 200, 300]
s = pd.Series(data)
```

```
print(s)
```

Cell 19 (code)

```
s = pd.Series([10, 20, 30], index=["a", "b", "c"])
print(s)
```

Cell 20 (code)

#Creating Datframes

Cell 21 (code)

#1. From List: Useful when you have data in row wise format
import pandas as pd

```
data = [
    ["Ali", 20],
    ["Sara", 22],
    ["John", 21]
]
```

```
df = pd.DataFrame(data, columns=["Name", "Age"])
print(df)
```

Cell 22 (code)

#2. Creating DataFrame from Dictionaries: Most Common, when you have column wise data
import pandas as pd

```
data = {
    "Name": ["Ali", "Sara", "John"],
    "Age": [20, 22, 21]
}
```

```
df = pd.DataFrame(data)
print(df)
```

Cell 23 (code)

3. Creating DataFrame from CSV File: For Real world datasets
import pandas as pd

```
df = pd.read_csv("climate_change.csv")
print(df.head(10))
```

Cell 24 (code)

#4. Creating DataFrame from Excel File: For Business Datasets
import pandas as pd

```
df = pd.read_excel("Water_Data.xlsx")
print(df.head())
```

Cell 25 (code)

Method	Input Type	Use Case
From List	Row-wise data	Small/manual data
From Dictionary	Column-wise data	Most common
From CSV	File	Real datasets
From Excel	Spreadsheet	Business data

Cell 26 (code)

#Reading Data (Loading Files)

Cell 27 (code)

#1. Read CSV File → read_csv()
import pandas as pd

```
df = pd.read_csv("climate_change.csv")
print(df.head())
```

Cell 28 (code)

#2. Read Excel File → read_excel()
import pandas as pd

```
df = pd.read_excel("Water_Data.xlsx")
print(df.head())
```

Cell 29 (code)

```
# Saving a Dataframe:
import pandas as pd

# Step 1: Create a DataFrame
data = {
    "Name": ["Ali", "Sara", "John"],
    "Age": [20, 22, 21],
    "Marks": [85, 90, 88]
}

df = pd.DataFrame(data)

# Step 2: Save to CSV/Excel file
df.to_csv("my_students.csv", index=False)
df.to_excel("my_students.xlsx", index=False)
print("File saved successfully!")
```

Cell 30 (code)

```
#View First Five Rows: head()
#View Last Five Rows: tail()
#Both can also accept a Number
#df.sample(number): returns random rows
#Dataset Info: df.info()
```

Cell 31 (code)

```
df = pd.read_excel("Water_Data.xlsx")
print(df.head(10))
print(df.sample(3))
print(df.info())
```

Cell 32 (code)

```
#Statistical Summary:
df.describe()
```

Cell 33 (code)

```
#Summary
| Function | Purpose |
| ----- | ----- |
| read_csv() | Load CSV file |
| read_excel() | Load Excel file |
| to_csv() | Save as CSV |
| to_excel() | Save as Excel |
| head() | First rows |
| tail() | Last rows |
| sample() | Random rows |
| info() | Structure info |
| describe() | Statistics |
```

Cell 34 (code)

```
#Select a single column with a name:
df["pH"]#Series
```

Cell 35 (code)

```
#Multiple columns: Returns a Dataframe  
df[["Temperature", "pH"]]
```

Cell 36 (code)

```
import pandas as pd  
  
data = {  
    "Name": ["Ali", "Sara", "John"],  
    "Marks": [85, 90, 88]  
}  
  
df = pd.DataFrame(data, index=["a", "b", "c"])  
print(df)
```

Cell 37 (code)

```
#loc: label based indexing  
df.loc['a']
```

Cell 38 (code)

```
df.loc['a', "Marks"]
```

Cell 39 (code)

```
df.loc['a', ["Name", "Marks"]]
```

Cell 40 (code)

```
df.loc['a':'b'] #Includes c
```

Cell 41 (code)

```
#Using iloc[] (Position-Based)  
df.iloc[0]
```

Cell 42 (code)

```
df.iloc[0, 0]
```

Cell 43 (code)

```
df.iloc[0:2] #2 is not inclusive
```

Cell 44 (code)

```
df.loc[0] # Error (no label '0')  
df.iloc["a"] # Error (expects integer)
```

Cell 45 (code)

#Data Filtering

Cell 46 (code)

```
import pandas as pd
df=pd.read_csv("sales_dataset.csv")
df.head()
```

Cell 47 (code)

```
df.columns
```

Cell 48 (code)

```
df[df["Customer"] > 1500]
```

Cell 49 (code)

```
df[df["Region"] == "West"]
```

Cell 50 (code)

```
df[(df["Sales"] > 1000) & (df["Region"] == "West")]
```

Cell 51 (code)

```
df[(df["Region"] == "South") | (df["Region"] == "East")]
```

Cell 52 (code)

```
df[df["Sales"] > 1500][["Customer", "Sales"]]
```

Cell 53 (code)

```
df.query("Sales > 1000 and Region == 'West'")
```

Cell 54 (code)

```
df.query("Sales > 1000 and (Region == 'West' or Region == 'South')")
```

Cell 55 (code)

```
df.query("`Sub Category` == 'Masalas'")
```

Cell 56 (code)

```
df[df["City"].isin(["Chennai", "Vellore"])]
df[df["City"]=="Chennai"| df["City"]=="Vellore"]
```

Cell 57 (code)

```
df[df["Sales"].between(1000, 2000)]
df[df["Sales"]>1000 & df["Sales"]<=2000]
```

Cell 58 (code)

```
df[df["Customer"].str.contains("a", case=False)]
```

Cell 59 (code)

```
df.info()
```

Cell 60 (code)

```
df["Order Date"] = pd.to_datetime(df["Order Date"])
```

```
df[(df["Order Date"].dt.year == 2017) & (df["Order Date"].dt.day == 5)]
```

Cell 61 (code)

```
df.nlargest(3, "Profit")[["Category", "Sub Category"]]
```

Cell 62 (code)

```
df.nsmallest(5, "Profit")
```

Cell 63 (code)

```
df.sort_values(by="Discount", ascending=False, inplace=True )
```

Cell 64 (code)

```
df.head()
```

Cell 65 (code)**Cell 66 (code)**

```
#inplace, ascending
```

Cell 67 (code)

```
df
```

Cell 68 (code)

```
#Add a new column with same value for all
```

Cell 69 (code)

```
df["Status"] = "Completed"
```

Cell 70 (code)

```
df["Status"]
```

Cell 71 (code)


```
#Add a new column from an existing column
df["Profit Ratio"] = df["Profit"] / df["Sales"]
```

Cell 72 (code)

```
df["Profit Ratio"]
```

Cell 73 (code)

```
df.insert(2, "Tax", df["Sales"] * 0.05)
```

Cell 74 (code)

```
df.drop("Discount", axis=1, inplace=True)
```

Cell 75 (code)

```
#Remove First Row
df.drop(0, inplace=True)
```

Cell 76 (code)

```
#Remove Specific Rows
df.drop([0, 1, 2], inplace=True)
```

Cell 77 (code)

```
#Data Cleaning in Pandas
```

Cell 78 (code)

```
#3.1 Check Missing Values
df=pd.read_csv("sales_dataset.csv")
print(df.isnull().sum())
```

Cell 79 (code)

```
# Drop rows with any missing value
df_cleaned = df.dropna()
```

Cell 80 (code)

```
df_cleaned.isna().sum()
```

Cell 81 (code)

```
df["Sales"].fillna(df["Sales"].mean())
```

Cell 82 (code)

```
df["Category"].fillna(df["Category"].mode()[0])
```

Cell 83 (code)

```
df["Date"].fillna(method='ffill', inplace=True)
```

Cell 84 (code)

```
print(df.duplicated().sum())
```

Cell 85 (code)

```
df.drop_duplicates(inplace=True)
```

Cell 86 (code)

```
df.head()
```

Cell 87 (code)

```
df['Sub Category'] = df['Sub Category'].str.lower()
```

Cell 88 (code)

```
df['Sub Category']
```

Cell 89 (code)

```
df['Sub Category'] = df['Sub Category'].replace({'masalas': 'spices'})
df['Sub Category']
```

Cell 90 (code)

```
#Detect Noisy values, then can use replace function
# Check unique values
print(df['Category'].unique())
```

```
# Detect typos or inconsistent entries
df['Profit'].value_counts()
```

Cell 91 (code)

```
# Example: Increase all prices by 10%
df['Profit'] = df['Profit'].apply(lambda x: x * 1.1)
```

Cell 92 (code)

```
df['Profit']
```

Cell 93 (code)

```
# Add profit_margin based on profit
df['profit_margin'] = df['Profit'].apply(lambda x: 'High' if x > 1000 else 'Low')
df['profit_margin'].value_counts()
```

Cell 94 (code)

```
df['Region'].value_counts()
```

Cell 95 (code)

```
df['Region_code'] = df['Region'].map({'West': 0, 'Central': 1, 'East': 2, 'South':3, 'North':4})
```

Cell 96 (code)

```
df['Region_code']
```

Cell 97 (code)

```
df[['Region_code', 'Region']]
```

Cell 98 (raw)

```
df.head()
```

Cell 99 (code)

```
#Groupby
```

Cell 100 (code)

```
df.groupby('Category')['Profit'].value_counts()
```

Cell 101 (code)

```
df.groupby('Category')['Profit'].mean()
```

Cell 102 (code)

```
df.groupby('Category')['Sales'].count()
```

Cell 103 (code)

```
df.groupby('Category')['Profit'].max()
```

Cell 104 (code)

```
df.groupby('Sub Category')['Profit'].min()
```

Cell 105 (code)

```
df.groupby('Category').agg({  
    'Profit': 'sum',  
    'Sales': 'mean',  
    'Discount': 'min'  
})
```

Cell 106 (code)

```
# Total sales per product  
pivot = pd.pivot_table(df,  
    index='Category',  
    values='Profit',  
    aggfunc='sum')
```

```
print(pivot)
```

Cell 107 (code)

```
pivot = pd.pivot_table(df,  
index='Category',  
values='Profit',  
aggfunc=['sum','mean'])  
print(pivot)
```

Cell 108 (code)

```
pivot = pd.pivot_table(df,  
index='Category',  
values=['Profit','Sales'],  
aggfunc={'Profit':'mean','Sales':'min'})  
print(pivot)
```

Cell 109 (code)

```
# Total sales per product per month  
pivot = pd.pivot_table(df,  
index='Category',  
columns='Order Date',  
values='Profit',  
aggfunc='sum',  
fill_value=0) # Replace NaN with 0  
print(pivot)
```